

Plano e Matriz de Segurança da API

Lucca Carneiro Della Pasqua - Análise e Desenvolvimento de Sistemas

Sistema de controle de estoque.

Esse é o plano completo que reúne as decisões de segurança que irão orientar a implantação da API do sistema de estoque da empresa DDA Metalúrgica, utilizando o Spring Security. O plano foi desenvolvido de acordo com as características reais do projeto, sendo algumas delas: autenticação via convite(não é possível realizar um cadastro por um caminho público), histórico de movimentações do estoque imutáveis e dois tipos de perfis de acesso(operador e gestor).

1 - Perfil de Acesso

No sistema da DDA Metalúrgica, não existe cadastro público. Todo novo usuário precisa ser convidado por um gestor.

Perfil	Descrição
Público	Pode realizar requisições sem autenticação. Pode realizar login e aceitar um convite.
Operador	Colaborador do setor do estoque. Trabalha no dia a dia registrando entradas e saídas do sistema. Consulta o estoque, acompanha o painel, visualiza e resolve os alertas.
Gestor	Alem de tudo o que o Operador realiza, o gestor cadastra produtos, fornecedores, fabricantes. Define estoque mínimo, gerencia solicitações de compra, emite relatórios, convida novos usuários e gerencia convites.

2 - Matriz de segurança

Recursos	Ameaças	Possível Impacto	Controle preventivo adotado
Login (POST /auth/login)	Força bruta.	Acesso não autorizado à conta de um usuário.	- Senhas com hash BCrypt. - Limites de tentativa. - Mensagem de erro genérica que não dê nenhuma pista a um possível atacante.
Aceite de convite (POST /user/invite)	A adivinhação de um token ou a reutilização de um token já utilizado.	Criação indevida de contas de acesso não autorizadas no sistema.	Para a prevenção, será utilizado um token grande(UUID), com prazo de expiração. O convite se torna inválido após o aceite ou a revogação.

Gestão de convites (POST /user/invite)	Escalonamento de privilégios, a possibilidade de um usuário sem permissão conseguir criar contas ou gestores.	Contas com privilégios indevidos podem ser criadas.	Autorização por papel do usuário, apenas Gestores conseguem criar ou revogar convites. O papel do convidado, é também definido pelo Gestor.
Endpoints protegidos	Acesso sem autenticação.	Visualização ou alteração indevida de informações.	Validação por JWT em todas as rotas. Por padrão, a política é negar. Deny-by-default no Spring Security.
Cadastro (POST, PUT de produtos, fornecedores, fabricantes).	SQL Injection.	Leitura, alteração ou destruição de dados de forma indevida.	Consultas parametrizadas via Spring Data JPA. Validação de entrada com o Bean Validation.
Campos de texto livre	Scripts armazenados no banco de dados e acionados depois.	Execução de códigos no navegador de outro usuário.	Resposta sempre no formato application/json. Sanitização de entrada de dados.
Operações de escrita (POST, PUT, DELETE)	CSRF (Cross-Site Request Forgery).	Execuções de ações indevidas em nome de um usuário autenticado. Ocorre por meio do roubo de cookies por exemplo.	Autenticação via JWT e não por cookies. Essa implantação torna a API imune ao CSRF clássico, assim podemos desabilitar a proteção por CSRF.
Acesso via navegador	CORS mal configurado. Sites maliciosos consumindo a API.	Vazamentos de dados ou uso indevido da API por meio do navegador do usuário	CORS restrito à origem do front-end oficial.
Histórico de movimentação	Adulteração do histórico (edição ou exclusão de movimentação).	Perda de rastreabilidade e de confiança do saldo. A auditoria fica comprometida.	Tabela apenas permite inserção. API não expõe PUT e DELETE de movimentações.
Corpo das requisições	Mass assignment, quando o cliente envia campos que não deveria como: current_quantity, role ou id.	Burlar as regras de negócio, alterando saldo ou se autopromover a gestor.	DTOs de entrada separados dos de saída.
Notificações (GET /notification/{id})	Acessar recursos de outros usuários trocando o id.	Ler notificações destinadas a outro usuário.	Consultas sempre filtradas pelo usuário autenticado.

Respostas da API	Exposição de dados sensíveis.	Vazamento de credenciais	O password_hash nunca é retornado. O token de convite é exposto apenas uma vez, na criação.
Relatórios e listagens	Consumo excessivo de recursos	Lentidão ou indisponibilidade do sistema	Paginação, rate limiting e acesso restrito ao gestor.

3 - Plano de Segurança da API

3.1. Autenticação

A autenticação dos usuários será realizada utilizando token JWT, de tal forma que o servidor não irá manter sessão. O fluxo:

1. Usuário envia email e senha para POST /auth/login
2. O servidor valida as credenciais do cliente, comparando a senha com o hash do BCrypt e retorna um token JWT assinado e com um prazo de validade.
3. Nas requisições seguintes que o usuário realizar, o cliente enviará o token no cabeçalho. O Spring Security valida a assinatura e a validade do token antes de a requisição se quer chegar ao controller.

3.2. Recursos acessíveis sem autenticação

Existem apenas dois recursos em que um usuário não autenticado consegue ter acesso, e são:

- POST /auth/login → Para obter token JWT
- POST /auth/accept-invite → Exige um token de convite

3.3. Recursos protegidos

Todos os demais endpoints exigem autenticação. Portanto, o cliente não conseguirá obter uma resposta bem sucedida ao chamar os endpoints: produtos, fornecedores, fabricantes, movimentações, alertas, notificações, solicitações de compra, relatórios, painel e gestão de usuários sem que ele esteja devidamente autenticado.

3.4. Controle de autorização

A autorização será baseada de acordo com o 'role' de cada usuário. Cada usuário possui um papel operador ou gestor. As restrições serão aplicadas em dois níveis, por rota onde a depender do papel do usuário, filtros de segurança serão aplicados e por método, onde o papel do usuário será pré requisito para chamar certos endpoints.

3.5. Armazenamento e proteção de credenciais

Senhas nunca são armazenadas em texto puro. São salvas como um hash usando o BCrypt. O password_hash como dito anteriormente nesse documento, nunca é exposto em nenhum endpoint. Os tokens de convite são aleatórios, de uso único e com uma data de expiração.

3.6. Configuração de CORS

O CORS(Cross-Origin Resource Sharing) será configurado para permitir requisições apenas da origem do front-end oficial. Junto a isso, serão analisadas as listas dos métodos e cabeçalhos permitidos. Origens desconhecidas serão bloqueadas.

3.7. Outros mecanismos

- SQL Injection: uso exclusivo do Spring Data JPA / consultas parametrizadas.
- Validação de entrada: todos os DTOs de entrada terão um Bean Validation que rejeitará dados malformados antes de passar pela camada que cuida das regras de negócio(service).
- Mass assignment: separação entre DTO de entrada e de saída, o cliente não consegue definir campos controlados pelo servidor.
- Integridade do histórico: transações são insert only. Não é possível editar uma transação passada.
- Rate limiting: no login e nos relatórios, para mitigar força bruta.
- Auditoria: cada movimentação registra o user_id de quem a realizou.

4 - Perfis e Permissões

Recursos / Operação	Público	Operador	Gestor
Realizar login	✓	✓	✓
Aceitar convite	🔑	-	-
Acessar o painel(dashboard)	✗	✓	✓
Consultar produtos e saldo de estoque	✗	✓	✓
Registrar movimentação	✗	✓	✓
Consultar histórico de movimentações	✗	✓	✓
Consultar e resolver alertas	✗	✓	✓
Consultar as próprias notificações	✗	✓	✓

Criar solicitação de compra	✗	✓	✓
Aprovar / alterar status de solicitação de compra	✗	✗	✓
Cadastrar / editar / excluir produto	✗	✗	✓
Definir estoque mínimo de um produto	✗	✗	✓
Cadastrar / editar / excluir fabricante	✗	✗	✓
Cadastrar / editar / excluir fornecedor	✗	✗	✓
Emitir relatórios (consumo / reposição)	✗	✗	✓
Convidar / gerenciar usuários e convites	✗	✗	✓
Listar usuários do sistema	✗	✗	✓